CH03 CSS样式与盒模型

HTML已经让我们能够为页面建立标题、段落、列表、链接、图片、表格和表单。浏览器打开这些文件时，内容通常能够显示，但页面的字号、颜色、宽度与间距还没有形成统一的设计。同样一份HTML，为什么换一份CSS就能呈现出不同的页面？设置了宽度以后，元素为什么仍可能超出容器？这些问题需要从样式规则与盒模型理解。

本章按照六个主题展开：**从HTML页面到样式页面、CSS规则与选择器、文字颜色与背景、层叠与继承、盒模型与正常文档流、校园活动页面实例**。前四部分解释样式怎样作用到元素，第五部分解释元素占据的空间，第六部分把这些知识组合成一个完整页面。

1 从HTML页面到样式页面

1.1 内容已经有了，为什么还需要CSS

下面的片段描述一则校园活动通知：

```
<section class="activity-card">
  <h2>校园读书交流会</h2>
  <p class="activity-meta">周五下午 · 图书馆交流空间</p>
  <p>带一本喜欢的书，与同学分享一次阅读体验。</p>
  <a class="action-link" href="#registration">查看参与方式</a>
</section>
```

即使没有编写CSS，浏览器也会把二级标题显示得较大、较粗，把链接显示为带下划线的文字，让段落之间存在一定间距。这些外观来自浏览器提供的默认样式。

HTML已经表达了内容关系：`section`组织一个主题区域，`h2`是这个区域的标题，`p`承载段落，`a`提供跳转。现在需要补充另一组规则：活动卡片应该有什么背景、文字离边框多远、不同内容怎样形成阅读层次。

```css
.activity-card {
  padding: 24px;
  color: #24324a;
  background-color: #ffffff;
  border: 1px solid #cbd5e1;
}
```

这条规则不会把`section`变成另一种语义元素，也不会修改活动名称。它控制符合选择条件的元素怎样显示。

技术	主要职责	一个具体例子
HTML	表达内容与关系	使用`h2`标记活动标题
CSS	控制视觉呈现与空间	为标题设置字号，为区域设置内边距
浏览器	解析并呈现规则的结果	计算文字换行、盒子尺寸和位置

把标题改成普通段落再放大字体，视觉上可能接近，但内容角色已经不同。选择HTML时先考虑含义，选择CSS时再考虑呈现。



图1-1 同一份HTML添加CSS前后的视觉对比

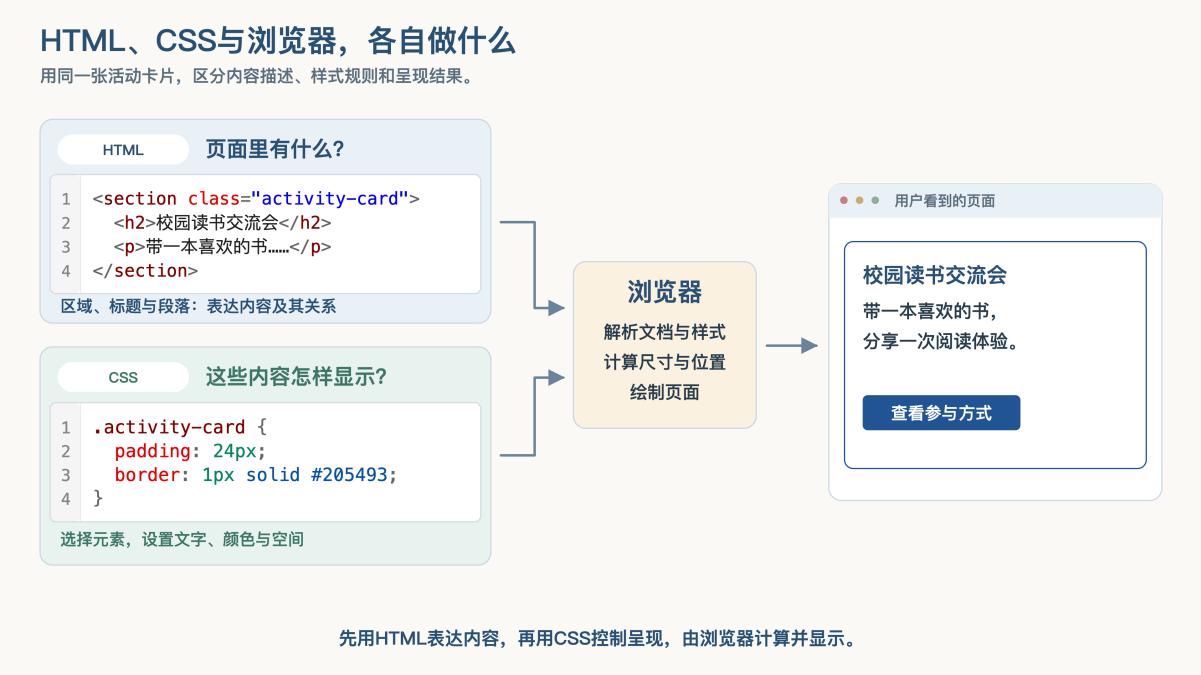


图1-2 HTML、CSS与浏览器的职责

1.2 CSS怎样参与页面呈现

CSS是Cascading Style Sheets的缩写，通常译为“层叠样式表”。可以用下面的简化过程理解浏览器的工作：

```
读取HTML，建立页面节点与内容关系
        ↓
读取CSS，判断哪些规则匹配哪些元素
        ↓
通过层叠与继承等规则确定属性值
        ↓
生成参与显示的盒子，计算尺寸与位置
        ↓
绘制文字、背景和边框等内容
```

这个简化过程包含三个主要问题：**选择器决定规则适用于谁，属性值决定怎样呈现，布局规则决定盒子怎样排列。**

不能把一个HTML元素机械地等同于一个固定大小的矩形。元素通常会生成参与布局的盒子；行内内容可能跨行形成多个片段，`display: none`则不生成布局盒子。第5部分将结合常见显示方式解释。



图1-3 浏览器呈现页面的简化过程

1.3 恢复第一次开发循环

准备一个项目文件夹，把HTML和CSS保存在其中。每次修改后，按照以下顺序确认结果：

1. 在编辑器中修改文件。
2. 保存文件。
3. 回到浏览器刷新页面。
4. 观察指定元素是否发生预期变化。

页面没有变化时，先核对编辑器与浏览器是否打开了同一份文件，再检查CSS有没有正确加载。临时调整十几个属性却不观察，很容易失去“哪次修改导致了哪种现象”的线索。

修改样式时，可以先预测，再改一个值，最后查看结果。例如，把一张卡片的`padding`从`16px`改成`32px`，先判断内容与边框之间的距离会变大还是变小。

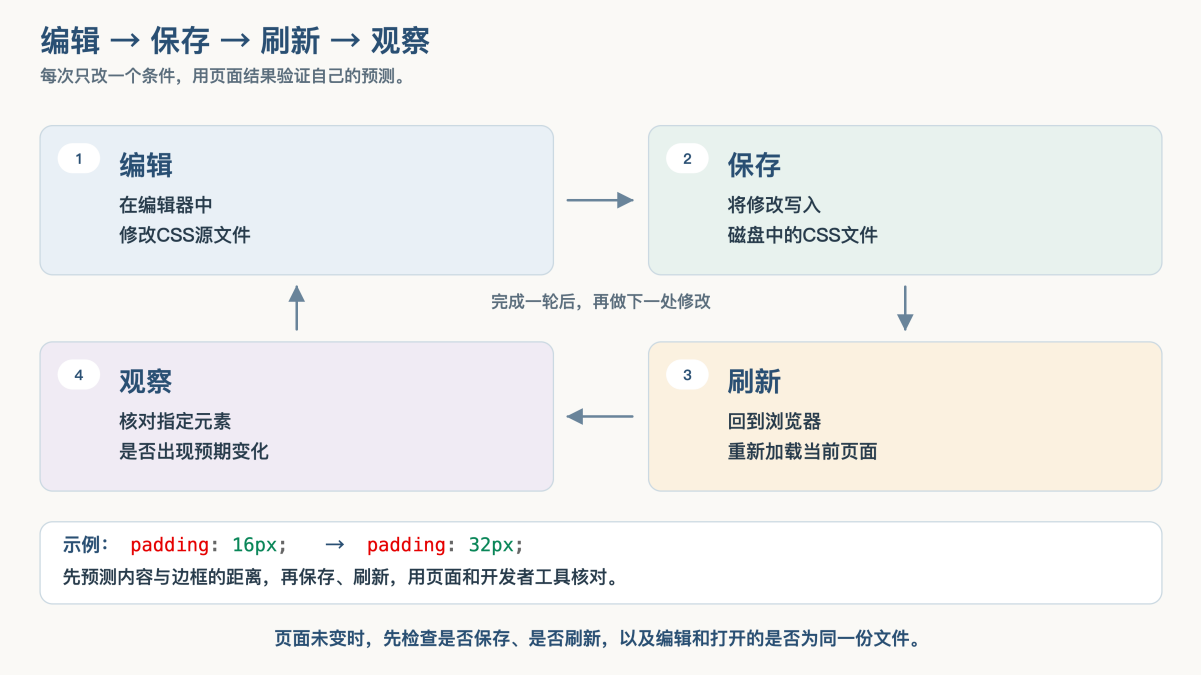


图1-4 编辑、保存、刷新与观察的开发循环

2 CSS规则与选择器

2.1 一条CSS规则怎样组成

```
.activity-card {
  color: #24324a;
  padding: 24px;
}
```

规则由选择器和声明块组成。`.activity-card`是选择器，花括号中的内容是声明块；每条声明又由属性和值组成。

部分	示例	含义
选择器	`.activity-card`	寻找具有这个类名的元素
属性	`color`	指明要设置文字颜色
值	`#24324a`	给出所采用的颜色
声明	`color: #24324a;`	一项完整的样式设置

属性和值之间使用英文冒号，不同声明之间使用英文分号。建议最后一条声明也写分号，方便以后继续增加属性。数字与单位之间不能加空格，例如应写`24px`，不能写`24 px`。

CSS注释使用`/* ... */`：

```
/* 内容与卡片边框之间保留空间。 */
.activity-card {
  padding: 24px;
}
```

CSS通常会忽略无法识别或无效的声明，继续处理其他有效内容。因此，某个属性拼错后，页面不一定整体失效。缺少分号、花括号或注释结束符，则可能影响更大范围。需要查看实际被浏览器接受的规则，而不只是检查有没有报错弹窗。



图2-1 CSS规则的组成

2.2 外部样式表与文件路径

示例项目采用以下目录：

```
src/ch03/
├── index.html
├── css/
│   └── style.css
└── images/
    └── activity.svg
```

在HTML的`head`中引用CSS：

```
<link rel="stylesheet" href="css/style.css">
```

`rel="stylesheet"`说明关联资源是样式表，`href`给出位置。对当前目录结构，浏览器从`index.html`所在目录出发，进入`css`文件夹，读取`style.css`。

`style.css`中只写CSS规则，不需要再包一层`<style>`标签。扩展名也应是`.css`，而不是`.css.txt`。

还要注意两种相对路径的起点：HTML中的`href`、`src`通常相对于当前HTML文档的URL解析；外部CSS中`url(...)`的路径相对于该CSS文件的URL解析。上述路径关系以没有设置`base`元素为前提。

例如，同一张图片在本项目中的写法分别是：

```
<img src="images/activity.svg" alt="展开的书与交流气泡的插画">
```

```
/* 若在css/style.css中引用背景图片，需要先回到上一级目录。 */
.banner {
  background-image: url("../images/activity.svg");
}
```

正文中有信息含义的图片仍应使用`img`及合适的替代文本。背景图片更适合装饰，不应成为传递重要文字信息的唯一方式。



图2-2 外部样式表与相对路径

2.3 内部样式与内联样式

外部样式表便于多个页面复用，也让内容与样式分别维护。另有两种常见写法。

内部样式表放在HTML的`style`元素中，适合单页小实验：

```
<head>
  <meta charset="UTF-8">
  <title>活动通知</title>
  <style>
    h2 {
      color: #205493;
    }
  </style>
</head>
```

内联样式写在某个元素的`style`属性中，只直接设置这个元素：

```
<h2 style="color: #205493;">校园读书交流会</h2>
```

三种写法都是CSS的应用方式。外部样式表便于集中查找、复用和维护规则。不能把“外部、内部、内联”简单理解为按名称排列的固定优先级；外部与内部规则发生冲突时，仍要根据匹配、重要性、优先级与出现顺序等条件判断，第4部分再展开。

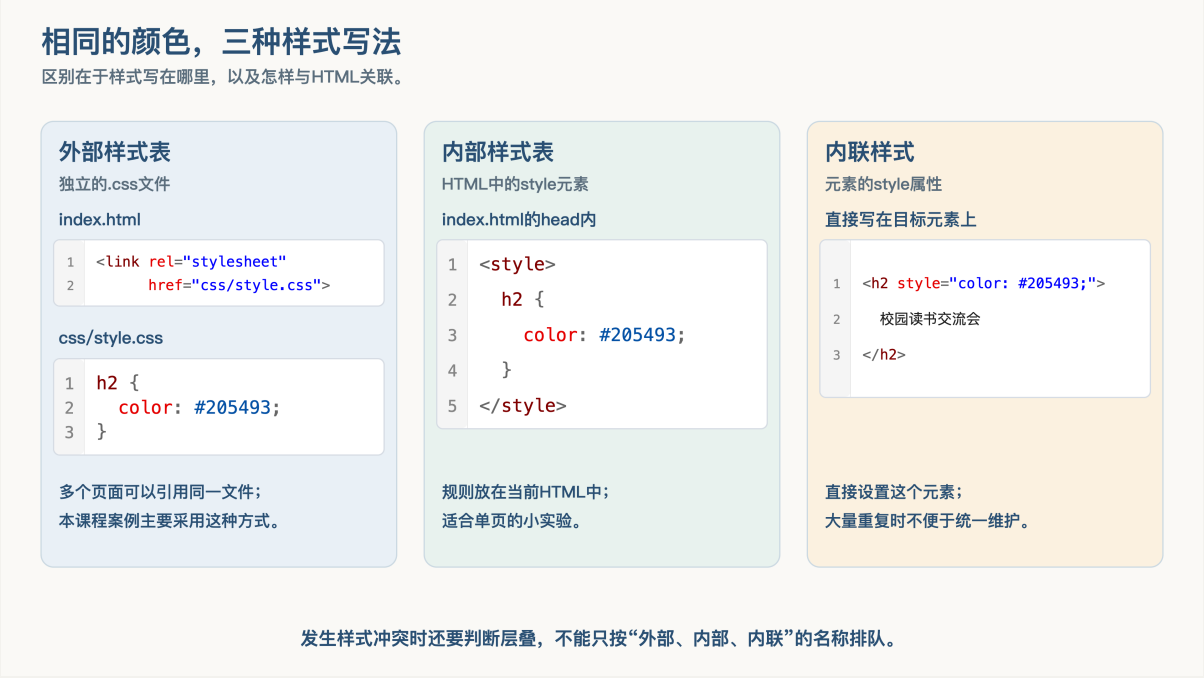


图2-3 CSS的三种引入方式

2.4 类型、类与ID选择器

类型选择器直接使用元素名称：

```
p {
  line-height: 1.7;
}
```

这条规则适用于所有匹配的`p`元素。若只想处理活动说明，不应依靠“它恰好是第三个段落”定位，而可以为内容命名。

```
<p class="activity-meta">周五下午 · 图书馆交流空间</p>
<p class="activity-meta">适合对象：对阅读与分享感兴趣的同学</p>
```

```
.activity-meta {
  color: #526174;
}
```

HTML中的类名不写点，CSS的类选择器前面要写点。一个类可以用于多个元素，一个元素也可以属于多个类：

```
<section class="activity-card notice-card">
  <h2>参与提醒</h2>
</section>
```

`.activity-card`可以设置卡片共有外观，`.notice-card`可以补充提醒区域的边框。同一个元素的多个类在HTML中的先后顺序，不决定冲突声明谁生效。

ID选择器前面使用`#`：

```
<section id="registration" class="activity-card">
  <h2>参与方式</h2>
</section>
```

```
#registration {
  border-color: #205493;
}
```

ID在同一文档中应唯一，也常作为页内链接的目标。类适合表达可复用样式，ID适合标识页面中的唯一位置，不宜仅为提高优先级而增加ID。

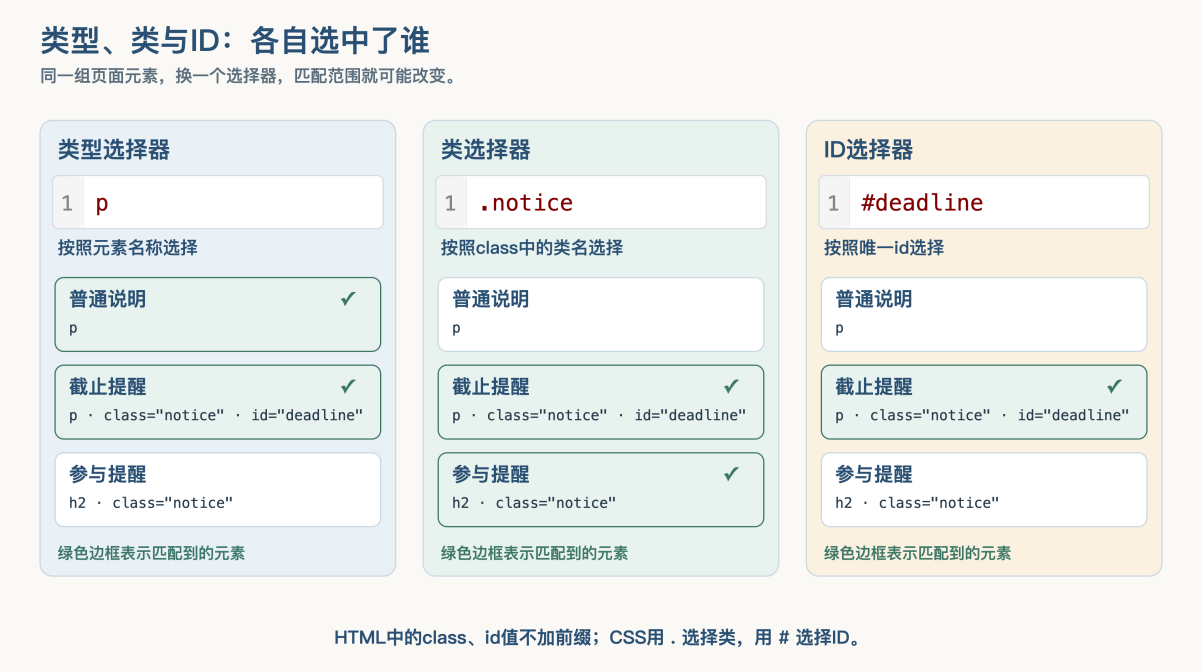


图2-4 类型、类与ID选择器

2.5 通用选择器与共同设置

通用选择器`*`匹配所有元素：

```
* {
  box-sizing: border-box;
}
```

这条规则直接匹配每个元素，让它们使用同一种尺寸计算方式。`box-sizing`本身不是默认继承的属性，所以这与“只给body设置后让所有后代继承”不同。第5部分将解释它的含义。

通用选择器不意味着应该清除全部默认样式。把所有元素的`margin`与`padding`都设为零，会同时清除列表缩进、标题间距等已有帮助。初学时按实际需要调整，更容易理解变化来自哪里。

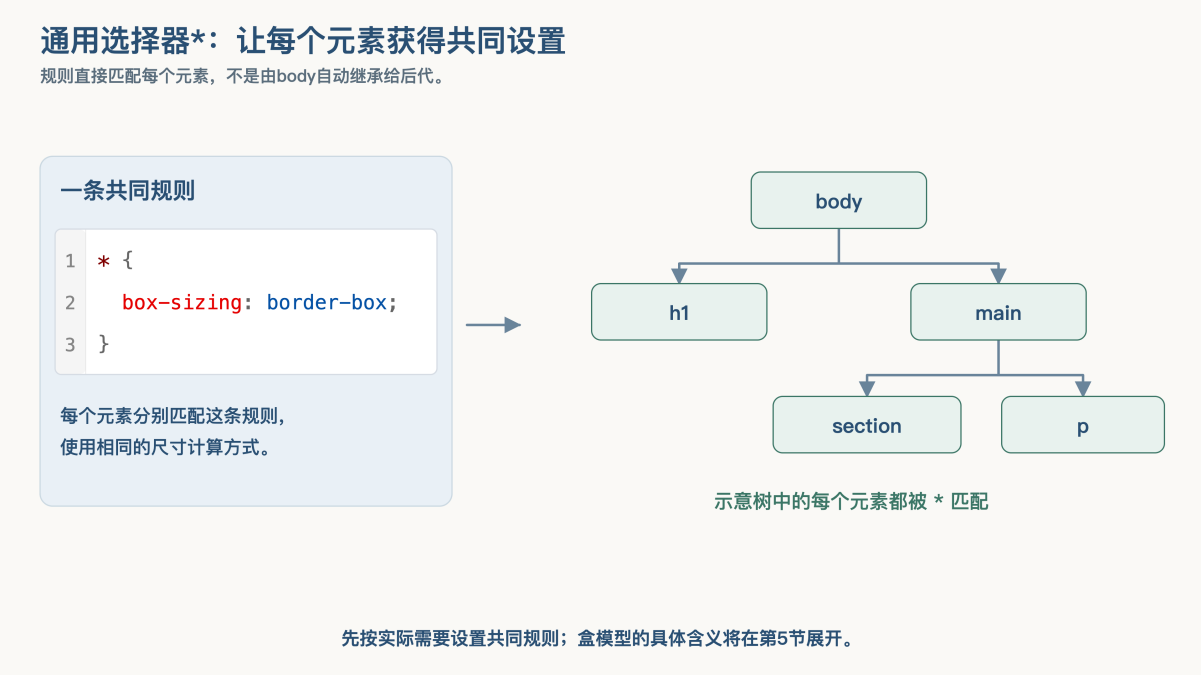


图2-5 通用选择器与共同设置

2.6 选择器列表、后代与子元素

以下HTML用于比较选择条件：

```
<section class="activity-card">
  <h2>活动介绍</h2>
  <p>欢迎阅读通知。</p>
  <div class="note">
    <p>这里是一条补充说明。</p>
  </div>
</section>
```

逗号表达分别选择。

```
h1,
h2 {
  color: #163a66;
}
```

这是一份选择器列表，分别匹配`h1`和`h2`。逗号不是父子关系符号。

空格表达后代关系。

```css
.activity-card p {
  color: #526174;
}
```

两个段落都是卡片的后代，都会匹配。后代可以相隔多层。

大于号表达直接子元素关系。

```css
.activity-card > p {
  margin-bottom: 12px;
}
```

这次只匹配卡片的直接子段落；位于`.note`中的段落不是卡片的直接子元素。

还需区分`.activity-card p`与`.activity-card.note`：前者选择卡片内部的段落，后者选择同时具有`activity-card`和`note`两个类的同一个元素。阅读选择器时先明确“最终选中谁”，再看它前面附加了哪些条件。

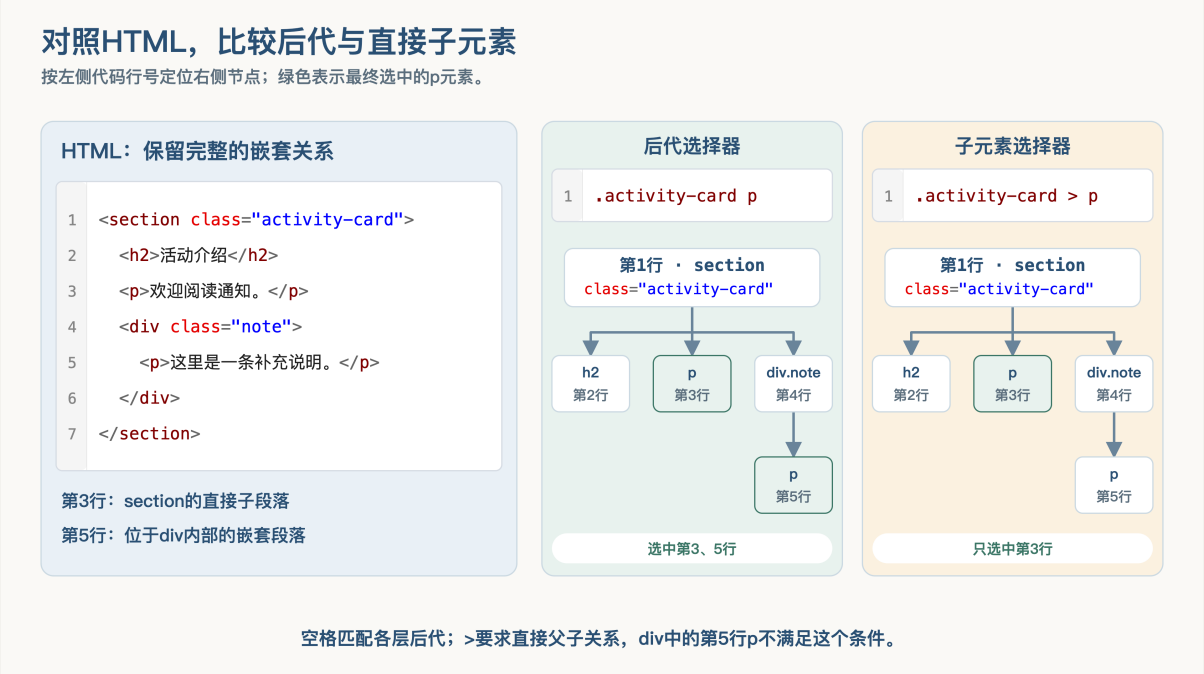


图2-6 后代与直接子元素



图2-7 逗号、空格与连写的区别

2.7 用伪类描述交互状态

链接在鼠标悬停时可以显示另一种背景：

```
.action-link:hover {
  background-color: #163a66;
}
```

`:hover`是伪类，用于匹配处于特定状态的元素。它不需要在HTML中写成`class="hover"`，状态由浏览器根据用户操作判断。

使用键盘按Tab键时，浏览器会依次把焦点移动到链接和表单控件等可聚焦元素。可以为需要明显焦点提示的状态设置样式：

```
.action-link:focus-visible {
  outline: 3px solid #92400e;
  outline-offset: 3px;
}
```

`:focus-visible`是否匹配，由浏览器结合获得焦点的方式等因素判断，键盘操作是常见场景。`outline`用于画出焦点轮廓，不占用盒模型中的边框空间。不能只设计悬停效果就认为键盘操作也已经得到照顾。



图2-8 伪类与链接的交互状态

2.8 从“没效果”回到选择条件

```
<p class="activity-meta">活动时间待通知。</p>
```

```
#activity-meta {
  color: #b42318;
}
```

类名与选择器里的文字相同，但`#`寻找的是ID，HTML提供的是类。当前声明并非“被别的规则覆盖”，而是根本没有匹配这个段落。

排查时区分三个层次：样式表有没有加载；规则有没有匹配；匹配以后声明有没有胜出。第一个问题涉及路径，第二个涉及选择条件，第三个才涉及层叠。



图2-9 样式失效的三个检查层次

3 文字、颜色与背景

3.1 从页面的共同文字规则开始

```
body {
  font-family: system-ui, "PingFang SC", "Microsoft YaHei", sans-serif;
  font-size: 16px;
  line-height: 1.7;
  color: #24324a;
}
```

`font-family`给出一组按顺序尝试的字体。`system-ui`使用系统界面字体，后面的字体名称与`sans-serif`提供后备选择。不同操作系统上实际使用的字体可能不同，因此换行位置不必完全一致。

`font-size`设置字号，`color`设置文字颜色。`line-height`控制行框高度，并影响多行文字的阅读间距。这里使用无单位的`1.7`，表示按当前元素字号的1.7倍计算；若正文为`16px`，相应值为`27.2px`。

这些共同设置让正文形成统一的阅读基准。标题、链接和表单控件仍可能有直接规则或浏览器默认样式，需要结合第4部分检查。表单控件可显式使用`font: inherit`取得周围文字的字体设置。



图3-1 页面的共同文字规则

3.2 字号、字重与行高

```
.page-title {
  font-size: 2rem;
  font-weight: 700;
  line-height: 1.25;
}

.activity-meta {
  font-size: 0.9375rem;
}
```

`font-weight`控制字重，常见值有`normal`、`bold`以及数值字重，字体实际可用的粗细仍取决于字体本身。

标题字号较大，行高可以比正文紧凑，但仍应允许长标题换行。内容区不宜为了保持截图高度就固定一个过小的`height`，否则标题变长或用户放大文字后可能溢出。

HTML标题级别表达内容层级，CSS字号表达外观。给`h2`设置比`h1`更大的字号，它也仍然是二级标题。



图3-2 字号、字重与行高

3.3 文本对齐与文字修饰

```
.site-header {
  text-align: center;
}

.action-link {
  text-decoration: none;
}
```

`text-align`控制块容器内部行内内容的对齐。把段落设成`text-align: center`，居中的是段落里的文字，不会因此让段落盒子变窄或在父容器中居中。

`text-decoration`可控制下划线等修饰。普通正文链接保留下划线，通常更容易识别。若把链接设计成明显的按钮外观，可以取消下划线，但仍要保留清晰的文字、形状与焦点提示。

实际跳转使用`a`，执行动作使用`button`。一个链接即使被CSS画成按钮，也仍然提供链接的语义和行为。



图3-3 文本对齐与链接修饰

3.4 文字颜色与背景颜色

```
.notice-card {
  color: #78350f;
  background-color: #fff7ed;
}
```

`color`用于文字等前景颜色，`background-color`用于元素背景。背景通常可以延伸到内容区、内边距及边框后方，外边距不属于本元素的背景绘制区域。

常见的颜色写法如下：

```
color: navy;
color: #205493;
color: rgb(32, 84, 147);
background-color: rgb(32 84 147 / 10%);
```

十六进制颜色的六位数字分别表示红、绿、蓝分量。最后一种写法给背景设置透明度；透明部分会与后面的背景共同产生视觉结果，不会让子元素文字一起透明。整个元素的`opacity`则会连同后代一起参与透明合成，二者不能互换。

选择颜色时要同时观察文字与背景。提醒信息还应有“参与提醒”等文字说明，不能只靠红色边框表达重要性。

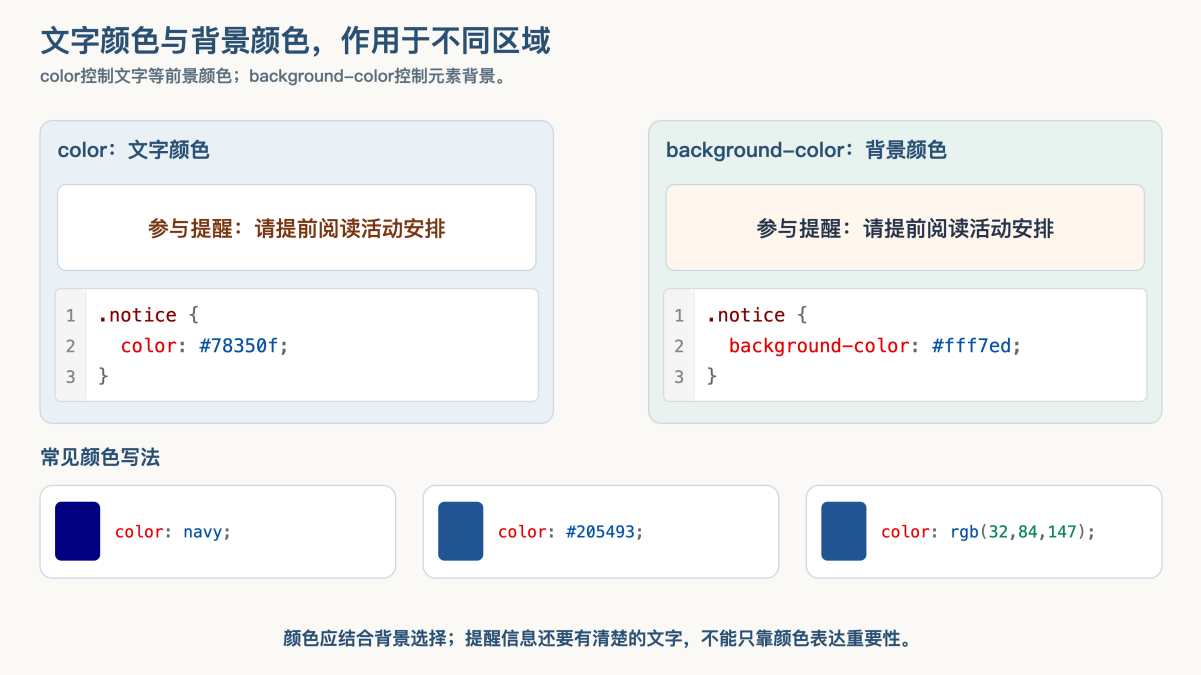


图3-4 文字颜色与背景颜色



图3-5 背景透明度与整体透明度

3.5 长度单位需要参照什么

写法	参照关系	常见用途
`16px`	CSS像素；不保证等于一个物理屏幕像素	细边框、明确的局部尺寸
`1rem`	根元素`html`的字号	字号与统一间距
`1em`	与字号有关，具体参照取决于属性	让间距随局部字号变化
`90%`	具体参照取决于属性	例如`width`通常参照包含块宽度
`0`	长度为零时可以省略单位	清除默认边距

`em`不能只背成“相对于父元素”。在`font-size`中，`em`参照继承得到的字号，通常来自父元素；在`padding`等属性中，它参照当前元素自己的计算字号。

```
html {
  font-size: 16px;
}

.note {
  font-size: 1.25rem;
  padding: 1em;
}
```

在这个明确设定下，`.note`的字号是`20px`，四边内边距也是`20px`。若把`padding`改为`1rem`，内边距就是`16px`。

字号如果写成百分比，参照的是继承字号；宽度写成百分比，通常参照包含块宽度。遇到不确定的属性，应查阅其百分比参照的定义。

完整案例不固定`html`字号，保留浏览器的根字号设置。不能无条件认为所有浏览器里的`1rem`永远等于`16px`。



图3-6 长度单位的参照关系



图3-7 em与rem的计算对照

3.6 从属性堆叠到阅读层次

一张活动页至少要让读者快速分辨页面名称、活动内容、时间地点和操作入口。可以用少量规则建立层次：

内容	主要处理	原因
页面主标题	较大字号、较强字重	帮助确认页面主题
区域标题	稍大字号、统一颜色	区分内容区域
正文	合理行高、稳定文字颜色	保证连续阅读
时间地点	稍小字号、较次要颜色	表达辅助信息
操作链接	明显边框或背景、焦点提示	帮助识别和操作

不必为每段文字设置不同的颜色和字号。规则越多，后面判断覆盖关系与维护一致性时的成本也越高。



图3-8 用文字样式建立阅读层次

4 样式为什么这样生效：层叠与继承

4.1 多条规则可以共同设置一个元素

```
<p class="activity-meta">周五下午 · 图书馆交流空间</p>
```

```css
p {
  line-height: 1.7;
}

.activity-meta {
  color: #526174;
}
```

两条规则都匹配同一个段落，但设置的是不同属性。段落同时获得相应行高与文字颜色，不需要在两条规则之间“只选一条”。

层叠主要处理的是某个元素、某个属性存在多个候选声明时怎样选择。因此，浏览器可能从不同规则中取得不同属性，形成最终的样式组合。

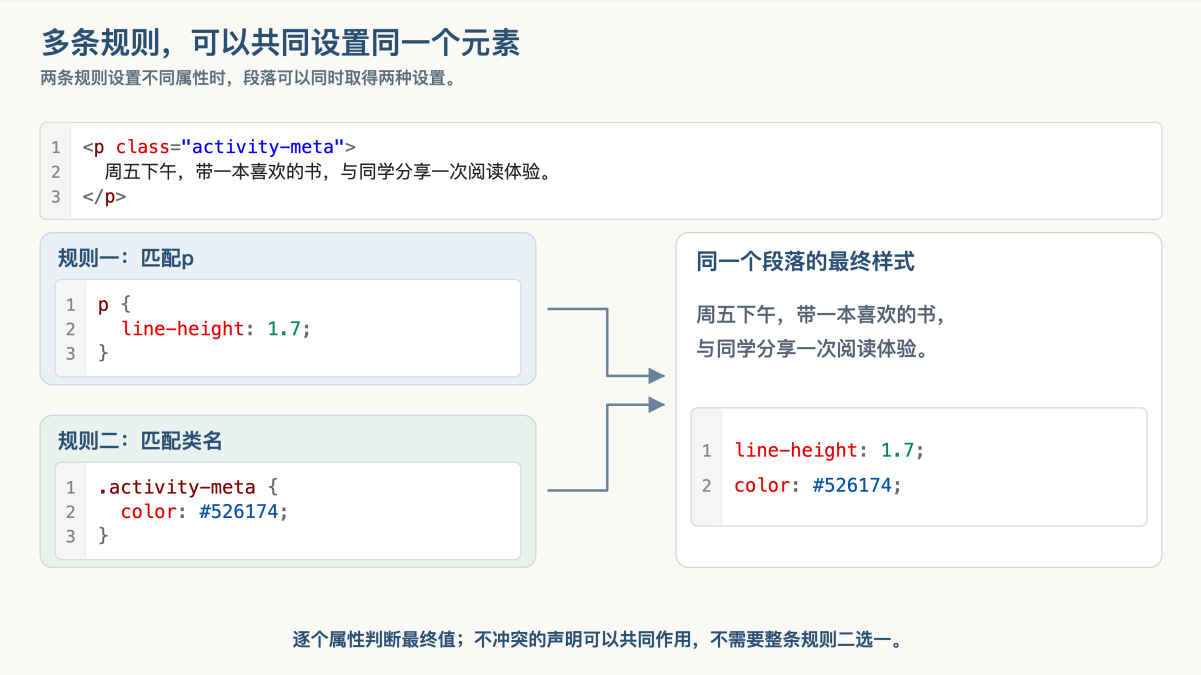


图4-1 多条规则共同设置一个元素

4.2 同等条件下再比较优先级

```html
<p id="deadline" class="activity-meta">请在周四前了解参与安排。</p>
```

```
#deadline {
  color: #b42318;
}

.activity-meta {
  color: #526174;
}

p {
  color: #24324a;
}
```

三条规则都直接设置这个段落的`color`。在本例中，它们同属作者编写的普通样式，没有层叠层、动画等其他因素。此时可以比较选择器的优先级，ID选择器高于类选择器，类选择器高于类型选择器，最终文字使用`#b42318`。

这不是“最后写的永远生效”。虽然`p`出现在后面，但在这个例子中，它的优先级更低。

常见选择器的优先级关系如下：

选择器	例子	比较方式
ID选择器	`#deadline`	高于单个类选择器
类、属性选择器与伪类	`.activity-meta`、`[required]`、`:hover`	同属一类优先级贡献
类型选择器与伪元素	`p`、`::before`	低于单个类选择器
通用选择器	`*`	本身不增加选择器优先级

复杂选择器的比较不能把不同类别数量简单相加，也不能把它们当成可以随意进位的十进制分数。遇到复杂冲突，可以先简化选择器，再利用开发者工具确认。



图4-2 同一属性的选择器优先级

4.3 优先级相同，再看出现顺序

```
.activity-meta {
  color: #526174;
}

.activity-meta {
  color: #205493;
}
```

两条规则的其他条件一致，选择器也相同，因此后面的`color`声明生效。

同样的规则可以位于多个外部样式表中，也可以分别位于外部样式表与HTML内部的`style`元素中。在相关条件相同时，比较的是文档中组织样式的顺序，而不是网络上哪个CSS文件先下载完。

只移动HTML里`class="activity-meta notice-text"`两个类名的顺序，不会改变CSS声明的顺序。



图4-3 优先级相同时比较出现顺序

4.4 内联样式与重要声明

```
<p class="activity-meta" style="color: purple;">参与安排待更新。</p>
```

在网页编写者设置的普通样式之间，内联声明通常优先于样式表中的普通声明。看到段落是紫色时，应同时检查元素的`style`属性。

`!important`是声明的重要性标记：

```
.activity-meta {
  color: #b42318 !important;
}
```

它可以使这个作者重要声明覆盖普通作者声明，包括本例中普通的内联颜色。但不能把它解释为“任何情况下都最高”；用户样式、其他重要声明以及完整层叠规则仍可能影响结果。

遇到冲突，先解释来源、匹配和覆盖关系，再决定是否需要修改规则。连续叠加ID或`!important`，会让后续修改更难预测。



图4-4 内联样式与重要声明

4.5 继承发生在父子之间

```
<section class="activity-card">
  <h2>活动介绍</h2>
  <p>欢迎参加校园读书交流会。</p>
</section>
```

```
.activity-card {
  color: #24324a;
  padding: 24px;
}
```

如果子元素没有其他生效的文字颜色声明，`h2`与`p`可以继承父元素的`color`。但它们不会自动各自获得`24px`内边距，因为`padding`默认不继承。

常见的可继承属性包括`color`、`font-family`、`font-size`、`line-height`和`text-align`；`width`、`height`、`margin`、`padding`、`border`、`background-color`默认不继承。

某个子元素看上去也有父元素的背景色，不一定说明背景发生了继承。子元素背景默认通常透明，父元素的背景可以透出来。可以给子元素增加一块白色背景，观察两者的区别。

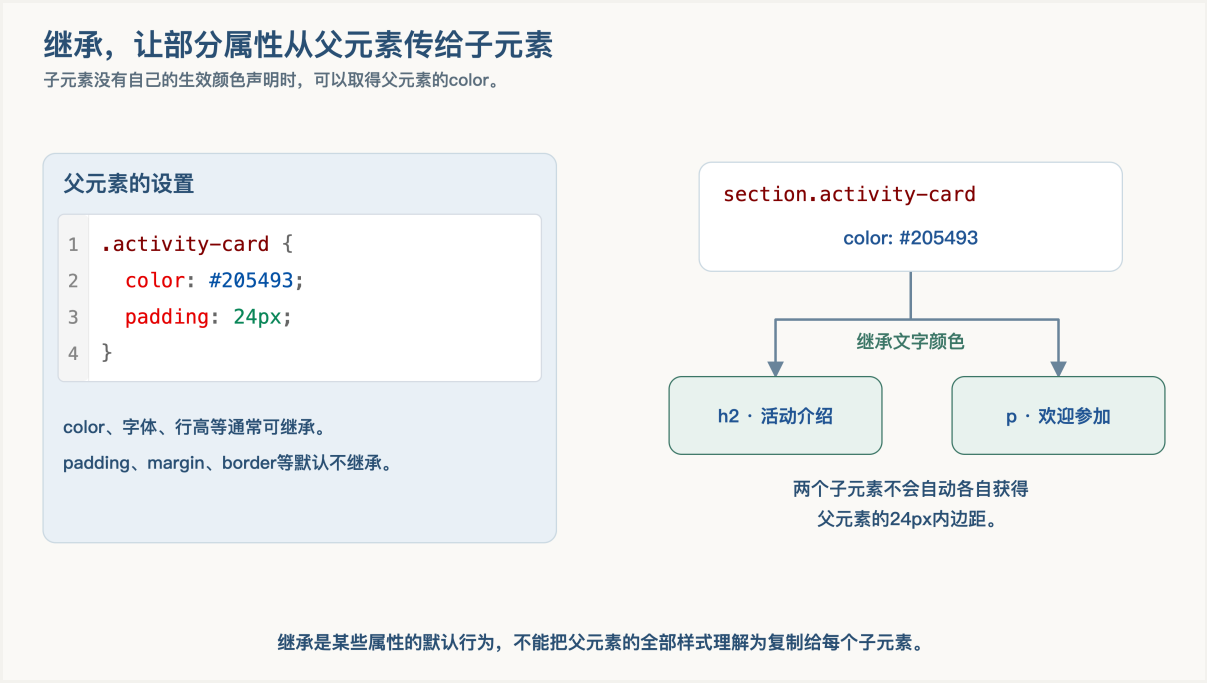


图4-5 继承发生在父子之间



图4-6 透明背景不等于背景继承

4.6 直接声明与继承值怎样区分

```
<section id="activity">
  <p>这段文字是什么颜色？</p>
</section>
```

```
#activity {
  color: #b42318;
}

p {
  color: #205493;
}
```

段落最终为蓝色。`#activity`虽然使用ID选择器，但它设置的是父元素；段落已经有直接适用的颜色声明，不再采用父元素继承下来的红色。

这里不是“类型选择器打败了ID选择器”。两条规则没有在同一个元素上直接竞争同一属性。**继承值不与当前元素的直接声明按选择器优先级同台比较。**

反过来，删掉`p`上的颜色声明，段落才会取得父元素的红色。这种对照比只背“ID大于类、类大于标签”更能解释实际页面。



图4-7 直接声明与继承值

4.7 浏览器默认样式也是来源之一

页面即使没有作者样式，仍可能存在：`body`四周的默认留白、标题的字号与上下边距、列表的标记与缩进、链接的颜色和下划线。

```css
body {
  margin: 0;
}
```

这条声明直接重设`body`外边距。它不会清除标题和段落自己的边距，也不会让列表标记消失。

链接尤其适合观察继承边界。给`body`设置文字颜色后，链接可能仍采用浏览器为链接直接设置的颜色。若希望它明确跟随所在区域的文字颜色，可以写：

```css
.site-nav a {
  color: inherit;
}
```

`inherit`显式要求该属性取父元素的计算值。是否要这样设置，应结合链接可识别性决定。



图4-8 浏览器默认样式与局部重设



图4-9 inherit显式采用父元素的值

4.8 使用开发者工具解释结果

在浏览器中右键点击目标元素，选择“检查”。不同浏览器的界面名称可能略有差异，通常可以找到以下区域：

区域	观察内容	可以回答的问题
Elements / 元素	浏览器解析后的节点与属性	我选中的是不是目标段落？
Styles / 样式	匹配规则、文件位置、声明状态	哪条声明被覆盖？来自哪里？
Computed / 计算样式	最终计算值及相关来源	这个元素最终采用什么颜色和尺寸？
盒模型区域	内容区、内外边距和边框	哪一层占用了空间？

被划掉的声明往往表示未被采用，但还要留意无效值、属性不适用等提示，不能只凭一条删除线猜原因。

确认顺序可以固定为：找到目标元素，确认规则匹配，检查属性来源，暂时禁用一条声明，观察变化。工具里的临时改动通常不会自动保存到源文件，确认以后需要写回CSS并保存。



图4-10 用开发者工具解释最终样式

5 盒模型、尺寸与正常文档流

5.1 从文字外面的空间认识盒模型

一张卡片除了文字，还需要留白与边界。以普通块盒子为例，从内到外可以区分四个区域：

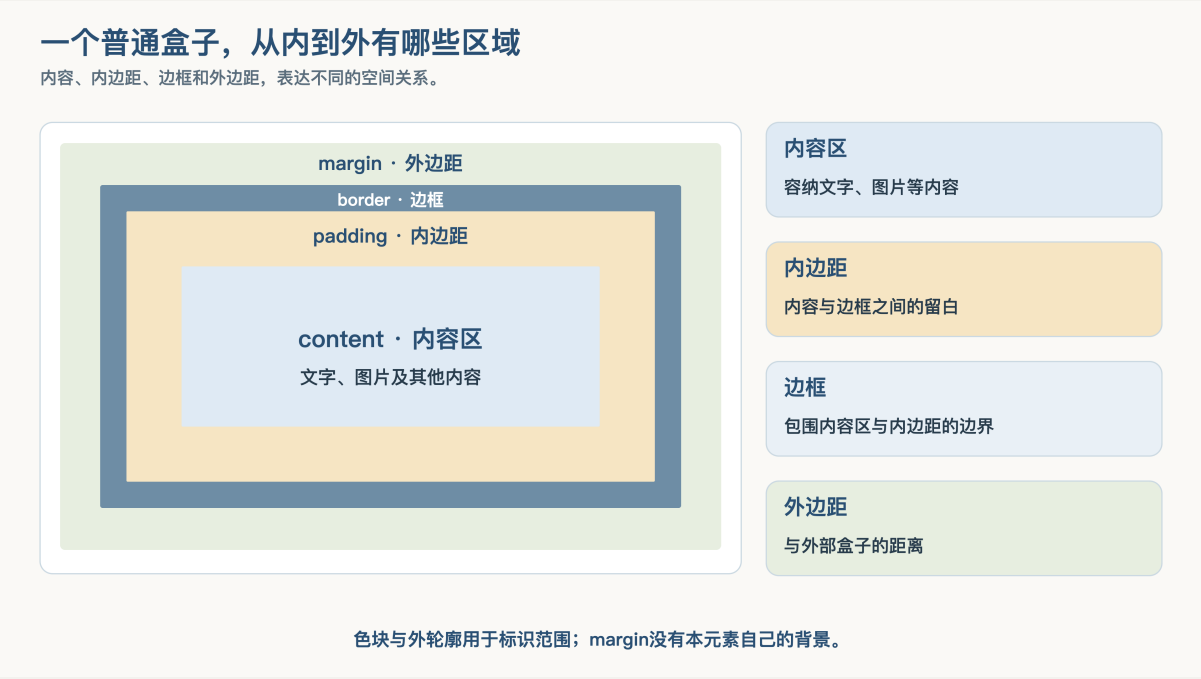


图5-1 盒模型的四个区域

区域	对应属性	活动卡片中的含义
内容区	`width`、`height`等参与确定	放置标题、段落和图片
内边距	`padding`	内容与卡片边框之间的留白
边框	`border`	卡片的边界线
外边距	`margin`	卡片与前后其他区域之间的距离

背景可以显示在内边距区域；外边距没有本元素自己的背景。想让文字离蓝色卡片边缘更远，通常调整`padding`；想让两张卡片之间更远，通常调整`margin`。

5.2 宽度与高度究竟设置了什么

```
<div class="measure-box">测量盒子</div>
```

```
.measure-box {
  box-sizing: content-box;
  width: 240px;
  height: 100px;
  padding: 20px;
  border: 2px solid #205493;
  margin: 10px;
}
```

说明一个盒子的尺寸时，需要说清测量的边界。`width`和`height`的含义与`box-sizing`有关：在本例明确采用的`content-box`模式下，它们设置内容区的宽度与高度。

内容区是`240px × 100px`。左右各增加`20px`内边距与`2px`边框，边框外沿宽度为`240 + 20 + 20 + 2 + 2 = 284px`；上下采用相同的计算方法，边框外沿高度为`100 + 20 + 20 + 2 + 2 = 144px`。外边距位于边框之外，不包括在这两个结果中。

本例还设置了四边各`10px`的外边距。若把这个独立盒子的外边距范围也标出来，水平方向为`284 + 10 + 10 = 304px`，垂直方向为`144 + 10 + 10 = 164px`。因此可以依次区分三组尺寸：内容区`240px × 100px`、边框外沿`284px × 144px`、包含外边距的范围`304px × 164px`。

这里标记的是单个盒子的外边距范围；相邻盒子之间的实际距离还受布局规则影响，例如第5.11节将讨论的垂直外边距折叠，不能一律按两个外边距相加。

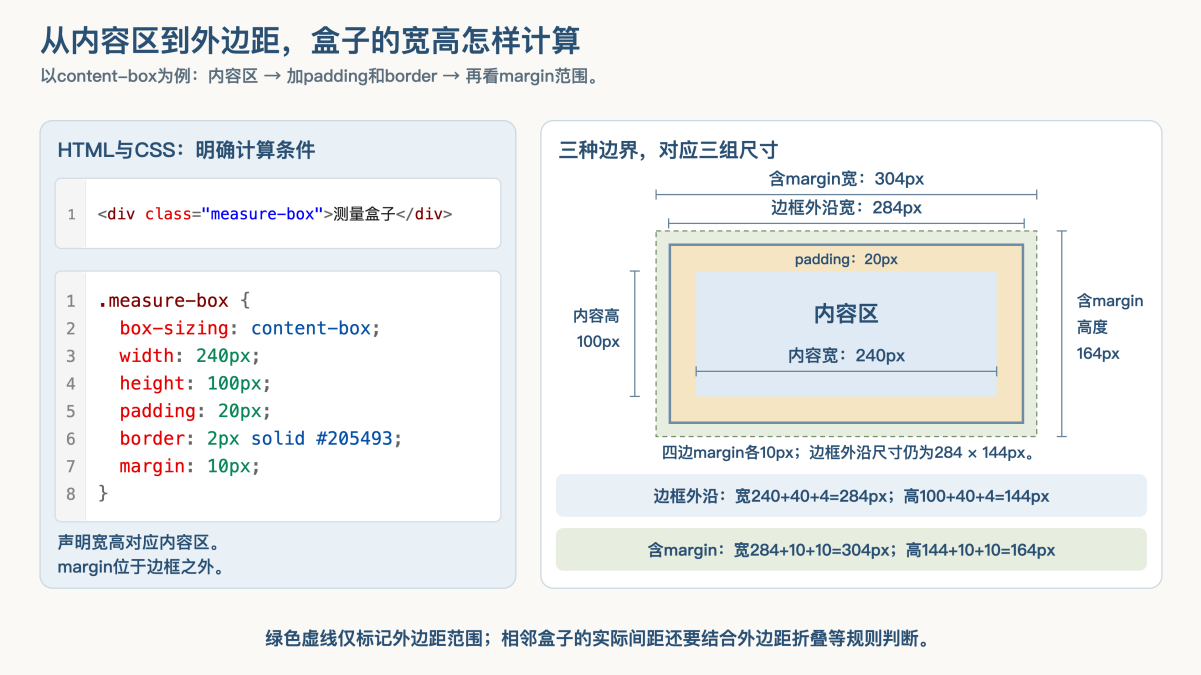


图5-2A 盒子宽度与高度的计算

如果移除上例的`height`声明，采用初始的`auto`，常见正常流块盒子的高度通常随内容计算。长段落换成多行，会让盒子向下增长。

```
.activity-description {
  width: 100%;
  height: 40px;
}
```

这类固定高度可能装不下换行后的文字。高度不足时，默认溢出行为往往仍让内容画到盒子外面，而不是自动把内容删掉。阅读型页面优先让内容决定高度；若需要保留最低空间，可以在具体需要时使用`min-height`。



图5-2B 自动高度与固定高度

5.3 内边距、边框与外边距

```
.activity-card {
  padding: 24px;
  border: 1px solid #cbd5e1;
  margin-bottom: 24px;
  border-radius: 12px;
}
```

这里的`padding`同时设置四边，`margin-bottom`只设置下方外边距。`border`简写一次给出宽度、线型和颜色；只写`border-width`而没有可显示的边框线型，不一定能看到边框。

`border-radius`让边框转角呈圆角，不会自动给其他内容增加内边距。圆角与留白是不同的设计问题。

分别设置某一方向时，可以使用`padding-top`、`margin-left`、`border-bottom`等属性。上述属性使用上下左右的物理方向。CSS还提供适合不同书写方向的逻辑属性。



图5-3 padding与margin的不同作用

5.4 内外边距的简写规则

写法	上	右	下	左
`margin: 16px;`	16px	16px	16px	16px
`margin: 16px 24px;`	16px	24px	16px	24px
`margin: 8px 16px 24px;`	8px	16px	24px	16px
`margin: 8px 12px 16px 20px;`	8px	12px	16px	20px

四值写法从上方开始，按顺时针方向排列。`padding`采用相同的四方向简写规则。

简写是一次设置多个子属性，而不是只补充“写出来的某个方向”。下面两条声明在顺序改变后，结果可能不同：

```
.activity-card {
  margin-left: 40px;
  margin: 16px;
}
```

后面的`margin`也设置了左外边距，因此最终左边是`16px`。如果需要在统一外边距之后单独改左边，应把`margin-left`放到后面。



图5-4 内外边距的简写规则

5.5 content-box：从内容区向外增加

明确把以下规则用于一个普通块盒子：

```
.measure-box {
  box-sizing: content-box;
  width: 240px;
  padding: 20px;
  border: 2px solid #205493;
  margin: 10px;
}
```

在没有额外尺寸限制等条件的这个例子中：

```
内容区宽度       = 240px
左右内边距之和 = 20px + 20px = 40px
左右边框之和     = 2px + 2px = 4px
边框外沿宽度     = 240px + 40px + 4px = 284px
左右外边距       = 10px + 10px
```

边框外沿宽度是`284px`。若只讨论这个独立盒子的水平外边距范围，再加两侧外边距得到`304px`；这不等于说所有布局的占用都可以机械相加，实际位置还受布局规则影响。

`margin`不包括在`width`中，也不包括在边框外沿宽度中。说明一个盒子“多宽”时，应说清测量的是内容区、边框外沿，还是连同外边距的范围。

5.6 border-box：在声明宽度内分配空间

只改变前一个例子的`box-sizing`：

```
.measure-box {
  box-sizing: border-box;
  width: 240px;
  padding: 20px;
  border: 2px solid #205493;
  margin: 10px;
}
```

此时声明宽度包含内容区、左右内边距和左右边框：

```
边框外沿宽度 = 240px
内容区宽度    = 240px - 40px - 4px = 196px
左右外边距仍分别为10px，不包含在240px中
```

这一模式不会把`padding`和`border`取消，而是在声明宽度内部给它们分配空间。内容区变窄后，文字可能更早换行，从而使自动计算的高度增加。

案例统一采用：

```
* {
  box-sizing: border-box;
}
```

它让后续的宽度推理更直接。如果项目还使用`::before`、`::after`生成内容，可以用`*, *::before, *::after`把伪元素也纳入统一设置。

上述减法示例的前提是声明尺寸足够容纳边框与内边距。若把宽度设得比它们之和还小，内容区不能变成负值，盒子仍可能大于声明宽度。

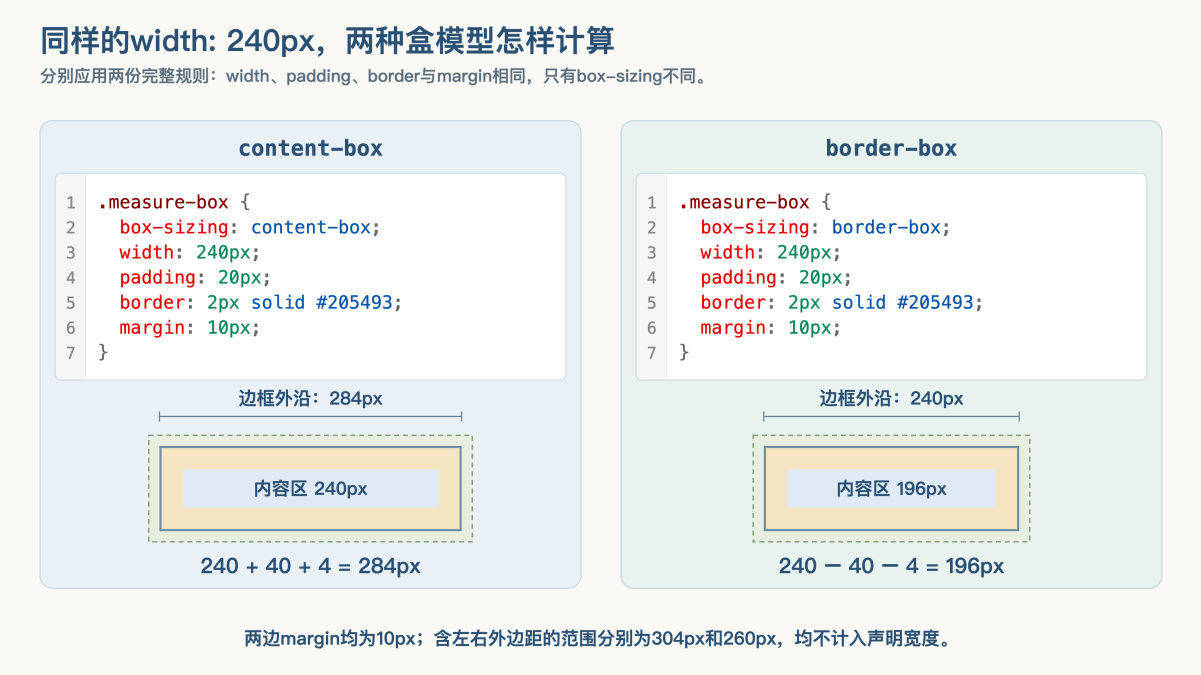


图5-5 两种盒模型的宽度对照

5.7 为什么width: 100%有时会溢出

假定父容器内容区宽度为`400px`：

```
.child-box {
  box-sizing: content-box;
  width: 100%;
  padding: 20px;
  border: 2px solid #205493;
}
```

`width: 100%`先让子盒子内容区取`400px`，再加内边距和边框，边框外沿达到`444px`。于是它超出了父容器的内容区。

使用`border-box`，同样的声明宽度会把内边距与边框包括进去，边框外沿保持为`400px`。但若再额外设置很大的水平外边距，仍可能溢出，因为外边距不在这个宽度里。

还有一个常被忽略的区别：普通正常流块盒子的`width: auto`不等于`width: 100%`。在常见的无定位块布局中，`auto`会按包含块的可用空间，结合边距、边框和内边距计算宽度，通常可以在添加内边距后继续装进父容器。不要给每个块元素都机械增加`width: 100%`。

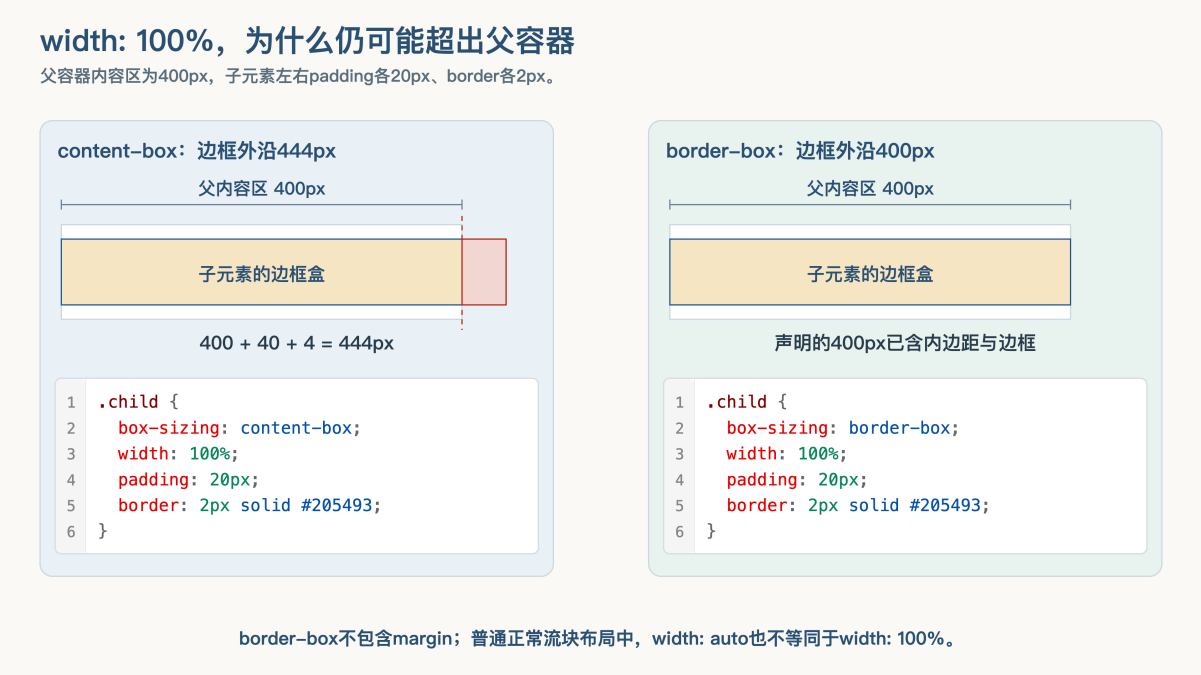


图5-6 百分比宽度与溢出

5.8 限制阅读宽度与水平居中

```
.page-content {
  width: 90%;
  max-width: 960px;
  margin: 24px auto;
}
```

在本例普通块布局中，`width`先让内容区域随包含块宽度变化，`max-width`限制它不要在宽屏上无限增长。若可用宽度大于盒子最终宽度，左右`auto`外边距分配剩余空间，使盒子居中。

假设父容器内容区为`1200px`，90%为`1080px`，受最大宽度限制后盒子宽度为`960px`，剩余`240px`分到左右，每侧`120px`。若父容器为`400px`，90%得到`360px`，没有超过上限，左右各留`20px`。这里假设盒子没有额外的水平外部限制，并采用前文的统一盒模型。

如果盒子本来就占满可用宽度，两侧没有剩余空间，`margin: auto`不会凭空产生可见的居中留白。它也不是任何布局中通用的垂直居中方式。



图5-7 最大宽度与水平居中

5.9 图片尺寸与内容比例

```
.activity-image {
  display: block;
  max-width: 100%;
  height: auto;
}
```

`max-width: 100%`限制图片不要比所在内容区域更宽；图片原本较小时，仅使用最大宽度不会强行把它放大。`height: auto`配合宽度变化，通常按图片自身比例计算高度，避免拉伸变形。

这里把图片设置为块盒子，便于单独占一行，也消除它默认参与行内排版时常见的基线下方留隙。`img`是替换元素，尺寸行为与普通行内文字不同，不能用“所有行内元素都不能设宽高”概括。

完整案例使用本地SVG图片，使页面离线也可打开。图片表达的内容由`alt`说明，CSS只负责它的显示尺寸与外观。



图5-8 图片尺寸与内容比例

5.10 正常文档流中的块与行内

在没有主动采用其他布局方式时，浏览器会按正常文档流组织内容。对横向书写的页面，可以观察两类行为：

- 普通块盒子通常上下排列，在自动宽度时使用可用的水平空间。
- 行内内容在行内依次排列，空间不足时换到下一行。

```
<p>读书会面向<strong>所有感兴趣的同学</strong>开放。</p>
<p>请先阅读<a href="#registration">参与方式</a>。</p>
```

两个段落默认上下排列，`strong`与`a`则进入所在段落的文字行。浏览器不是把HTML代码中的每一行都画成页面上的一行。

`display`可以改变显示行为，但不会改变内容语义：

值	典型行为	示例用途
`block`	在正常流中按块参与排列，可设置宽高	图片单独占一行
`inline`	普通文本盒子参与行内排版，通常不采用指定宽高	段落中的强调文字
`inline-block`	整体参与行内排列，内部可像块容器组织内容，可设置宽高	带内边距的导航链接
`none`	不生成布局盒子，相应空间也不保留	暂时隐藏一个区域

普通非替换行内元素仍然可以有背景、边框与内边距，但垂直方向的这些绘制通常不会把相邻文字行按同样幅度推开，容易与其他行重叠；其上下外边距也不按普通块盒子的方式影响行距。需要稳定的按钮外观时，通常使用`inline-block`，而不是据此记成“行内元素不能设置内边距”。

`display: none`通常也会让内容退出辅助技术的读取与交互范围。需要用户操作的链接和控件，不应为了解决视觉问题就随意隐藏。



图5-9 正常文档流与display

5.11 两个外边距为什么不总是相加

```
<div class="flow-demo">
  <p class="first">第一段内容</p>
  <p class="second">第二段内容</p>
</div>
```

```
.flow-demo {
  padding: 1px;
}

p {
  margin: 0;
}

.first {
  margin-bottom: 30px;
}

.second {
  margin-top: 20px;
}
```

例子先清除段落的默认外边距，再分别设置需要观察的两个方向。这样可以排除默认间距的干扰。

在这个简单正常流例子中，两个相邻块盒子的正向垂直外边距发生折叠，段落边框盒之间的距离是较大的`30px`，不是`50px`。

折叠并非所有方向、所有布局都存在。水平外边距不按这种方式折叠；Flex布局与Grid布局的项目之间也不发生这里的垂直外边距折叠。父元素与首尾子元素、空块等还存在其他折叠情形，详见附录B。

实际检查时，先确认规则确实生效，再观察布局机制。若把“声明被覆盖”和“外边距折叠”混在一起，就会对同一现象给出错误解释。



图5-10 相邻块盒子的外边距折叠

5.12 通过工具核对计算

选中第5.5节的`.measure-box`，检查Styles中的`box-sizing`以及盒模型区域。先记录内容区，再向外核对内边距与边框。

项目	CONTENT-BOX示例	BORDER-BOX示例
声明的width	240px	240px
内容区宽度	240px	196px
左右padding	各20px	各20px
左右border	各2px	各2px
边框外沿宽度	284px	240px
左右margin	各10px	各10px

只改变`box-sizing`后，观察文字是否增加换行。再把`padding`改成`30px`，重新计算并验证。

开发者工具里的`width`显示位置不同，可能表达计算后的CSS属性值或内容区尺寸，不能看到一个数字就直接称为“整个盒子的宽度”。应结合属性名、盒模型图和实际边界一起判断。

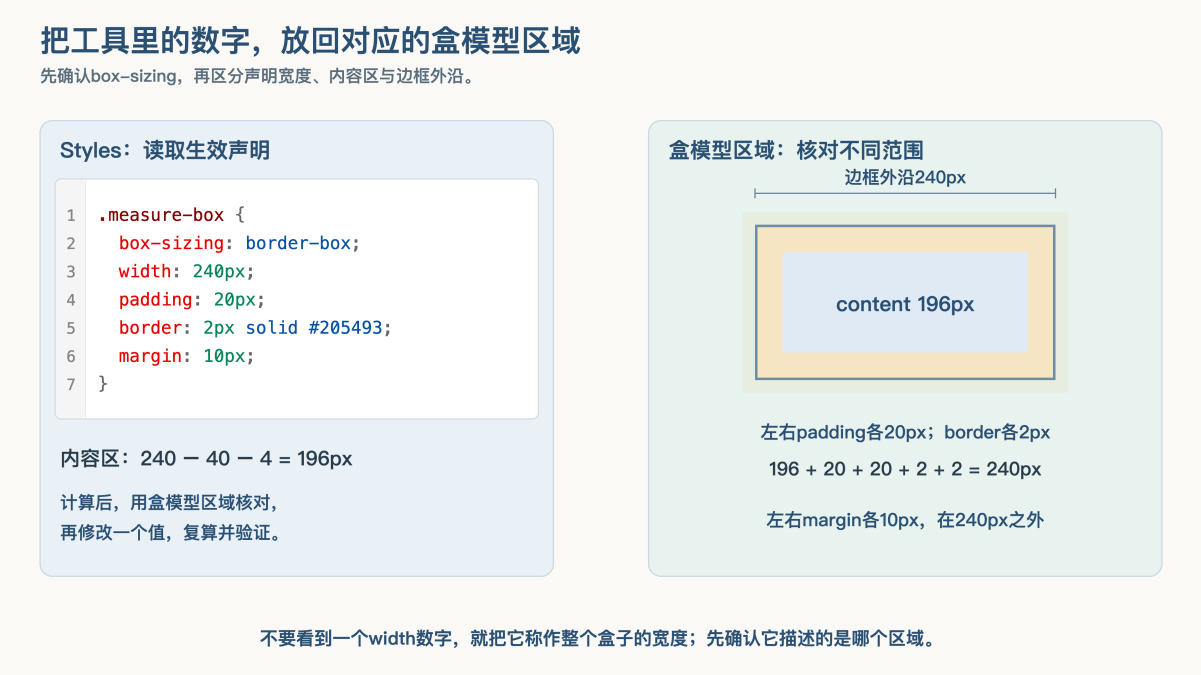


图5-11 用开发者工具核对盒模型计算

6 为校园活动页面添加样式

6.1 页面内容与功能

案例是一张校园读书交流会通知页，包含页头、导航、活动介绍、活动安排、参与提醒和参与方式。文字、地点与活动安排均为教学示例，不是正式活动通知。

页面按照HTML内容顺序上下排列，用正常流完成内容卡片，用`inline-block`处理导航与操作链接。它没有使用Flexbox、Grid、定位、多栏或媒体查询，也没有JavaScript。

页面上的“查看参与方式”与“返回页首”是页内链接。静态表单保留原生的输入与校验能力，提交按钮明确处于禁用状态，表示当前示例不接收、不保存也不发送报名信息。样式完成不等于报名业务已经实现。

完整代码见附录A。配套目录中还提供`01-html.html`，与`index.html`使用同一内容但不引用外部CSS，可用于比较默认样式与作者样式。

6.2 从全局设置建立阅读基准

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, "PingFang SC", "Microsoft YaHei", sans-
serif;
  font-size: 1rem;
  line-height: 1.7;
  color: #24324a;
  background-color: #f1f5f9;
}
```

先统一尺寸计算方式，再给正文设置共同文字规则。去掉`body`默认外边距，让页头背景延伸到页面边缘；内容区的留白由后续规则单独表达。

`font-size: 1rem`跟随根元素字号，不会把用户设置强制改成固定的16px。正文与标题使用相对字号时，用户调整根字号也更容易带动页面变化。

6.3 页头与导航链接

```
.site-header {
  padding: 32px 20px;
  color: #ffffff;
  background-color: #205493;
  text-align: center;
}

.site-nav a {
  display: inline-block;
  margin: 4px;
  padding: 6px 12px;
  border: 1px solid #ffffff;
  border-radius: 6px;
  color: #ffffff;
  text-decoration: none;
}
```

页头使用内边距，让文字与背景区域的边缘保持距离。导航链接使用`inline-block`，既能够在同一行排列，又能形成完整的内边距和边框；空间不足时，链接自然换行。

链接颜色被直接设为白色，避免仅给父容器设置颜色后，链接仍采用浏览器的默认颜色。悬停和焦点状态分别由后面的伪类规则处理。

6.4 内容区与卡片留白

```
.page-content,
.site-footer {
  width: 90%;
  max-width: 960px;
  margin: 24px auto;
}

.activity-card {
  margin-bottom: 24px;
  padding: 24px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  background-color: #ffffff;
}
```

父内容区的宽度随窗口变化，并在宽屏达到上限。卡片没有再指定`width: 100%`，使用普通块布局的自动宽度填充所在区域。

卡片的`padding`保持内部文字与边框的距离，`margin-bottom`保持相邻卡片的距离。区域标题的上外边距设为零，避免默认标题边距和卡片留白共同造成不必要的顶部空隙。

6.5 给已有结构设置表格和表单外观

表格中的时间与内容已经由`table`、`caption`、`th`和`td`表达。CSS只设置边框、文字对齐和单元格内边距，不把表格用于整个页面布局。

表单中的`label`与控件通过`for`和`id`建立关系。样式可以让标签单独占一行、输入控件铺满可用宽度，但不能用视觉上的相邻代替HTML中的关联。

```
label {
  display: block;
  margin-bottom: 6px;
```

```
  font-weight: 700;
}

input,
select,
textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #64748b;
  border-radius: 6px;
  font: inherit;
}
```

在统一`border-box`的前提下，控件的100%宽度已经包含边框和内边距。这正是第5.7节尺寸分析在完整页面中的应用。

6.6 沿一处页面结果还原原因

选择“参与提醒”区域，按如下顺序解释：

1. HTML中的`class="activity-card notice-card"`让它匹配两个类选择器。
2. `.activity-card`提供共同边框、圆角、内外边距和白色背景。
3. `.notice-card`再设置提醒区域的背景与左边框。
4. 子标题通过直接规则取得蓝色，普通段落取得正文继承或自身规则设置的颜色。
5. `border-box`参与尺寸计算，左边框变宽时，自动宽度的盒子仍按可用空间计算。

接着选中输入框，核对它的边框外沿是否超过父容器内容区。最后用Tab键依次移动到导航、操作链接和表单控件，确认焦点清楚可见；禁用按钮不会成为正常可操作的提交入口。

6.7 从完整页面进入修改与排错

可以依次进行三次局部修改，每次都恢复到正确状态后再做下一次。

修改	预期现象	检查重点
把`css/style.css`误写为`style.css`	作者样式没有加载，页面退回默认外观	路径与文件位置
把`.notice-card`误写为`#notice-card`	普通卡片样式保留，提醒区域的特殊样式失效	类与ID是否匹配
将输入框改为content-box	100%内容宽度外再增加内边距与边框，发生溢出	盒模型与尺寸计算

这些现象对应不同原因。先描述哪个区域发生变化，再提出假设，通过工具取得证据，最后进行小范围修复。

本章小结

CSS通过选择器把声明作用到HTML元素。页面的最终样式来自多个来源，规则匹配以后，还要根据层叠、优先级、出现顺序和继承等机制确定属性值。分析结果时，应区分未匹配、被覆盖、继承与浏览器默认样式。

盒模型把普通盒子的空间分为内容区、内边距、边框和外边距。`box-sizing`决定声明宽度对应哪些区域；`border-box`包含内边距和边框，但不包含外边距。正常文档流决定本章页面的基本排列，块盒子与行内内容以不同方式参与布局。

样式调试需要能够说明原因。编辑源文件、运行页面、用工具检查结果，再把确认过的修改保存回源文件，构成可以重复使用的开发过程。CH04将在这些盒子和尺寸认识之上，用Flexbox组织多个元素的位置与空间，并用媒体查询处理不同屏幕条件。

复习与思考

1. HTML中已经有标题和段落，CSS还需要解决哪些问题？修改字号是否会改变标题的语义级别？
2. `index.html`位于项目根目录，CSS位于`css/style.css`，图片位于`images/activity.svg`。分别写出HTML中引入CSS、HTML中显示图片、CSS中引用这张图片的相对路径。
3. `.activity-card p`、`.activity-card > p`和`.activity-card.notice-card`分别选择什么？
4. `class="activity-meta notice-text"`交换两个类名的顺序，是否一定改变颜色？为什么？
5. 同一个段落同时匹配`#deadline`、`.activity-meta`和`p`，三者都是同等条件下的普通作者声明，谁设置的冲突颜色生效？同优先级时再看什么？
6. 父元素使用ID选择器设为红色，子段落被类型选择器直接设为蓝色，段落为什么

是蓝色？

7. 一个元素看起来有父元素的背景颜色，是否能据此判断`background-color`发生了继承？
8. `text-align: center`和左右`margin: auto`分别影响什么？
9. 在根字号明确为`16px`时，某元素`font-size: 1.5rem; padding: 1em;`，其字号与内边距分别是多少？如果改成`padding: 1rem`呢？
10. 普通块盒子在`content-box`下设置`width: 300px; padding: 20px; border: 2px solid;`，边框外沿宽度是多少？改成`border-box`以后，内容区宽度是多少？
11. 为什么`width: 100%`不保证子元素一定不会超出父容器？为什么外边距不能忽略？
12. 正常流中两个相邻块盒子的正向垂直外边距分别为30px和20px，为什么实际间距可能是30px？
13. 为什么普通行内文字与`img`不能用同一条“宽高是否生效”的口号概括？
14. 在开发者工具中改了颜色，刷新后恢复原状，原因是什么？
15. 本章页面有表单和按钮，是否就完成了能接收报名的Web应用？还缺少哪些行为？

常见误解与代码审查

常见说法	需要修正的认识
没有CSS，浏览器不会显示内容	浏览器有默认样式，HTML仍能表达结构并显示
CSS选择器越长，写法就越专业	选择器应准确且易维护，过长会增加覆盖与修改成本
class后面写哪个名字，哪个就优先	HTML类名顺序不决定样式冲突，需检查CSS层叠
内部样式总比外部样式优先	二者仍按相关层叠条件比较，文件位置不是全部依据
ID永远压过其他颜色	先确认是否作用于同一元素；继承不按这种口号比较
所有属性都向子元素继承	字体等常继承，尺寸、边距与背景等通常不继承
width就是盒子最外面的宽度	需看box-sizing、边框与内边距，外边距另算
border-box会让padding消失	padding仍存在，内容区可用空间相应减少
给元素设置100%就不会溢出	还需检查盒模型、外边距、内容自身最小尺寸等条件
margin和padding都是留白，可以互换	内部留白与外部距离影响的区域、背景和布局不同
设了margin就可以把所有间距相加	正常流中的垂直外边距可能折叠
行内元素没有盒模型	普通行内盒子也有背景、边框和内边距，但尺寸与行距行为不同
CSS规则变灰或被划掉，就一定要加!important	应先判断不匹配、无效、不适用还是被覆盖
页面看起来像按钮，功能就已经存在	CSS改变外观，实际语义和行为还要由HTML与程序表达

下面是一段需要审查的代码：

```
<section id="notice" class="activity-card">
  <p class="message">请阅读活动提醒。</p>
</section>
```

```
#notice {
  color: red;
}

#message {
  color: blue;
}
```

```
.activity-card {
  box-sizing: content-box;
  width: 100%;
  padding: 24px;
  border: 1px solid;
}
```

需要判断：哪条颜色规则没有匹配预期元素？段落可能采用什么颜色？假定父容器内容宽度固定为500px，卡片的边框外沿宽度是多少？应怎样修改选择器和尺寸规则，使含义与显示符合预期？参考分析见附录C。

课后微任务：为已有HTML页面建立样式

任务目标

使用本章知识，为已有个人页面或校园活动页面添加统一样式。保留原有语义结构，通过规则、文字层次和盒模型，让页面更容易阅读。

起始状态

可以使用自己的HTML页面，也可以复制本章配套案例后改写内容。若使用配套案例，可以保留原有HTML结构，修改CSS并核对显示结果。资料可以使用昵称与虚构信息。

必做要求

1. CSS放在独立文件中，通过正确的相对路径引用。
2. 至少使用类型选择器、类选择器和一种状态伪类。
3. 设置正文、主标题、区域标题和链接的阅读层次。
4. 使用背景、边框、内边距和外边距区分内容区域。
5. 使用`border-box`，用有限宽度与自动外边距处理内容区。
6. 页面若有图片，设置合适的尺寸限制，避免超过所在区域。
7. 对照开发者工具，解释一条规则的匹配和一个盒子的尺寸。
8. 制造并修复一次路径、选择器或盒模型问题，保留简短记录。

无需为了完成本任务提前使用Flexbox、Grid或JavaScript。不能通过缩小全站文字、隐藏溢出内容等方式掩盖问题；应先解释问题的实际来源。

AI使用与审查

可以请AI解释某条声明，或提出小范围修改建议。提问时提供相关HTML、CSS、预期结果与观察到的现象；采用修改前先阅读差异，采用后再打开页面验证。

能够生成一整份样式表，不代表已经理解了它。提交时应能指出哪些规则来自参考、哪些是自己调整的，并解释至少一处布局或尺寸选择。

提交证据与验收

提交页面文件及以下简短记录：

```
我检查的元素与选择器：
我预期看到的结果：
开发者工具显示的最终属性值：
盒模型的content、padding、border、margin：
我故意制造的问题：
定位与修复依据：
修复后的验证结果：
AI提供了什么帮助，我检查或修改了什么：
```

任选一处`padding`、类名或颜色声明进行修改，先预测结果，再运行并解释变化原因。

附录A 完整校园活动页面

A.1 目录与运行

配套完整案例位于src/ch03，入口为index.html。

```
src/ch03/
├── index.html          带本章CSS的完整页面
├── 01-html.html        相同内容，不引入额外样式表
├── css/
│   └── style.css
└── images/
    └── activity.svg
```

双击`index.html`可以直接查看，双击`01-html.html`可以观察浏览器默认外观。也可以在`src/ch03`目录中启动静态服务器：

```
python3 -m http.server 8763
```

随后打开`http://localhost:8763/`。本章本地资源不依赖网络请求接口，因此不需要后端服务；这里的静态服务器只是另一种访问资源的方式。

下面完整列出HTML与CSS。配套目录包含本地SVG插画，不需要依赖外部图片服务；若自行重建目录，也可以换成自己的本地图像并同步修改路径、尺寸与替代文本。

A.2 index.html

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>校园读书交流会 | CH03教学示例</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
  <header id="top" class="site-header">
    <p class="eyebrow">校园活动 · 教学示例</p>
    <h1 class="page-title">校园读书交流会</h1>
    <p>带一本喜欢的书，分享一次阅读体验。</p>
    <nav class="site-nav" aria-label="页面导航">
      <a href="#introduction">活动介绍</a>
      <a href="#schedule">活动安排</a>
      <a href="#registration">参与方式</a>
    </nav>
  </header>

  <main class="page-content">
    <section id="introduction" class="activity-card">
      <h2>活动介绍</h2>
      <p class="activity-meta">周五下午 · 图书馆交流空间</p>
      <p>选择一本你喜欢的书，准备一段简短的介绍。你可以分享书中的一个观点，也可
以讲述这本书与你的故事。</p>
      <figure>
        <img class="activity-image" src="images/activity.svg"
             alt="展开的书与交流气泡的插画" width="800" height="300">
        <figcaption>从一本书开始，让不同的想法在交流中相遇。</figcaption>
```

```
      </figure>
      <a class="action-link" href="#registration">查看参与方式</a>
    </section>

    <section id="schedule" class="activity-card">
      <h2>活动安排</h2>
      <p>所有时间和地点仅用于课堂示例，不代表正式活动安排。</p>
      <table>
        <caption>交流会流程</caption>
        <thead>
          <tr><th scope="col">时间</th><th scope="col">内容</th></tr>
        </thead>
        <tbody>
          <tr><th scope="row">14:00–14:10</th><td>相互认识，介绍带来的
书</td></tr>
          <tr><th scope="row">14:10–14:40</th><td>轮流分享阅读体验
</td></tr>
          <tr><th scope="row">14:40–15:00</th><td>自由交流与书目推荐
</td></tr>
        </tbody>
      </table>
    </section>

    <section class="activity-card notice-card">
      <h2>参与提醒</h2>
      <ul>
        <li>可以选择小说、科普、历史等自己感兴趣的书。</li>
        <li>分享内容控制在三分钟左右。</li>
        <li>尊重不同观点，认真倾听其他同学的介绍。</li>
      </ul>
      <p class="notice-text">请使用昵称，不要在练习页面填写真实联系方式。
</p>
    </section>

    <section id="registration" class="activity-card">
      <h2>参与方式</h2>
      <p id="form-note" class="activity-meta">以下是静态表单练习，不接
收、不保存也不发送报名信息。提交按钮暂不可用。</p>
      <form aria-describedby="form-note">
        <div class="form-field">
          <label for="nickname">练习昵称（必填）</label>
          <input id="nickname" name="nickname" type="text" required
```

```
                placeholder="例如：小南">
        </div>
        <div class="form-field">
          <label for="book-type">准备分享的书籍类别</label>
          <select id="book-type" name="bookType">
            <option value="fiction">文学与小说</option>
            <option value="science">科学与技术</option>
            <option value="history">历史与人文</option>
            <option value="other">其他</option>
          </select>
        </div>
        <div class="form-field">
          <label for="book-note">一句话介绍这本书</label>
          <textarea id="book-note" name="bookNote" rows="3"
                    placeholder="写下你最想分享的一点"></textarea>
        </div>
        <button type="submit" disabled>静态练习，暂不提交</button>
      </form>
    </section>
  </main>

  <footer class="site-footer">
    <p>Web程序设计 · CH03 CSS样式与盒模型</p>
    <a href="#top">返回页首</a>
  </footer>
</body>
</html>
```

A.3 css/style.css

```
/* 统一普通元素的尺寸计算方式。 */
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, "PingFang SC", "Microsoft YaHei", sans-
serif;
  font-size: 1rem;
  line-height: 1.7;
  color: #24324a;
```

```
  background-color: #f1f5f9;
}

a {
  color: #205493;
}

.site-header {
  padding: 32px 20px;
  color: #ffffff;
  background-color: #205493;
  text-align: center;
}

.eyebrow {
  margin: 0 0 8px;
  font-size: 0.875rem;
}

.page-title {
  margin: 0 0 12px;
  font-size: 2rem;
  line-height: 1.25;
}

.site-header > p {
  margin-top: 0;
}

.site-nav a {
  display: inline-block;
  margin: 4px;
  padding: 6px 12px;
  border: 1px solid #ffffff;
  border-radius: 6px;
  color: #ffffff;
  text-decoration: none;
}

.site-nav a:hover {
  background-color: #163a66;
}
```

```css
.page-content,
.site-footer {
  width: 90%;
  max-width: 960px;
  margin: 24px auto;
}

.activity-card {
  margin-bottom: 24px;
  padding: 24px;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  background-color: #ffffff;
}

.activity-card > h2 {
  margin-top: 0;
  margin-bottom: 12px;
  color: #163a66;
  font-size: 1.5rem;
  line-height: 1.35;
}

.activity-meta {
  color: #526174;
  font-size: 0.9375rem;
}

figure {
  margin: 20px 0;
}

.activity-image {
  display: block;
  max-width: 100%;
  height: auto;
  margin: 0 auto;
  border-radius: 8px;
}

figcaption {
```

```
  margin-top: 8px;
  color: #526174;
  font-size: 0.875rem;
  text-align: center;
}

.action-link {
  display: inline-block;
  padding: 8px 16px;
  border: 1px solid #205493;
  border-radius: 6px;
  color: #ffffff;
  background-color: #205493;
  text-decoration: none;
}

.action-link:hover {
  background-color: #163a66;
}

table {
  width: 100%;
  border-collapse: collapse;
}

caption {
  margin-bottom: 8px;
  text-align: left;
  font-weight: 700;
}

th,
td {
  padding: 10px;
  border: 1px solid #cbd5e1;
  text-align: left;
  overflow-wrap: anywhere;
}

thead {
  color: #ffffff;
  background-color: #205493;
```

```css
}

.notice-card {
  border-left: 6px solid #b45309;
  background-color: #fff7ed;
}

.notice-text {
  color: #78350f;
  font-weight: 700;
}

.form-field {
  margin-bottom: 16px;
}

label {
  display: block;
  margin-bottom: 6px;
  font-weight: 700;
}

input,
select,
textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #64748b;
  border-radius: 6px;
  color: #24324a;
  background-color: #ffffff;
  font: inherit;
}

textarea {
  resize: vertical;
}

button {
  padding: 10px 16px;
  border: 1px solid #64748b;
  border-radius: 6px;
```

```
  color: #334155;
  background-color: #e2e8f0;
  font: inherit;
}

button:disabled {
  cursor: not-allowed;
}

a:focus-visible,
input:focus-visible,
select:focus-visible,
textarea:focus-visible {
  outline: 3px solid #92400e;
  outline-offset: 3px;
}

.site-header a:focus-visible {
  outline-color: #ffffff;
}

.site-footer {
  color: #526174;
  text-align: center;
}

.site-footer p {
  margin: 0 0 8px;
}
```

A.4 阅读与修改顺序

先看HTML，确认每个类名对应什么内容；再读CSS中的共同设置、页头导航、内容卡片、图片表格和表单控件。最后检查悬停与焦点状态。

`border-collapse`用于合并表格相邻边框；`overflow-wrap: anywhere`允许必要时打断长词，减少狭窄单元格中的文字溢出；`resize: vertical`允许用户沿垂直方向调整文本框；`:disabled`匹配禁用控件。

在浏览器里逐项尝试：去掉一条边框、增加卡片内边距、改变内容区最大宽度、临时取消`border-box`。每次只改一个条件，记录结果，再恢复正确版本。

附录B 选择器速查与进一步阅读

B.1 本章常用选择器

选择器	匹配对象
`p`	所有p元素
`.notice`	class中包含notice的元素
`#registration`	id为registration的元素
`*`	所有元素
`h1, h2`	分别匹配h1和h2
`.card p`	card后代中的p
`.card > p`	card直接子元素中的p
`.card.notice`	同时具有card与notice两个类的元素
`a:hover`	鼠标等指向设备悬停时的链接
`a:focus-visible`	浏览器判定需要显著显示焦点的链接

B.2 其他选择条件（扩展）

写法	含义	注意点
`input[required]`	具有required属性的input	判断属性存在，不是读取真假字符串
`input[type="text"]`	type属性值为text的input	属性值也是选择条件
`.card + .card`	紧接在card后面的card兄弟元素	两者需要同级且紧邻
`.card ~ .card`	card后面符合条件的card兄弟元素	不要求紧邻
`li:first-child`	同时是父元素第一个元素子节点的li	不等于寻找任意位置的第一个li
`li:nth-child(2)`	同时是父元素第二个元素子节点的li	计数考虑同级其他元素类型
`.notice::before`	在相应元素内容前生成的伪元素	常配合content，不应承载唯一的重要信息

类名能够清楚表达内容职责时，优先采用简单类选择器。

B.3 常用属性与值速查

用途	属性	常见写法
字体与字号	font-family、font-size、font-weight	`font-size: 1rem`
行高与文字对齐	line-height、text-align	`line-height: 1.7`
文字与背景颜色	color、background-color	`color: #24324a`
尺寸限制	width、max-width、height	`max-width: 960px`
内容区内留白	padding	`padding: 24px`
外部距离	margin	`margin: 24px auto`
边框与圆角	border、border-radius	`border: 1px solid #cbd5e1`
尺寸计算	box-sizing	`box-sizing: border-box`
显示行为	display	`display: inline-block`
焦点轮廓	outline、outline-offset	`outline: 3px solid #92400e`

B.4 外边距折叠的其他情形（扩展）

正常流中的垂直外边距除相邻兄弟之间外，还可能在父元素与首尾子元素之间、没有内容和边框内边距等隔开的空块中折叠。添加相应方向的边框或内边距，可以改变部分父子折叠条件；不能把所有间距异常都归结为同一种情况。

第5.11节只讨论两个正值。存在负外边距时不能继续套用“总取最大值”：正负值混合时要结合最大的正值与最小的负值，均为负值时采用最小的负值。遇到包含负外边距的代码，应结合折叠规则并通过工具验证。

B.5 其他层叠因素（扩展）

完整层叠还考虑声明来源、重要性、层叠层、动画与过渡等因素。选择器优先级是在相应前置条件确定后才参与比较的一个环节。本文的优先级例子有意限制在普通作者规则中，不应据此把“ID大于类”推广成所有场景的完整算法。

分析包含`@layer`、动画或第三方样式库的页面时，还需要结合这些因素判断最终结果。

附录C 自测参考与实践分析

C.1 核心问题参考要点

1. CSS控制视觉与空间；字号变化不改变HTML标题级别。
2. HTML引用CSS为`css/style.css`，HTML引用图片为`images/activity.svg`，该外部CSS中的图片URL为`../images/activity.svg`。
3. 后代段落、直接子段落、同时具有两个类的同一个元素。
4. 不一定，类名顺序不是CSS声明顺序，不参与决定冲突声明胜出。
5. 在题设条件下ID选择器胜出；优先级相同时再比较出现顺序。
6. 子段落已有直接声明，不再取父元素的继承颜色，不是在两种选择器之间直接竞争。
7. 不能，透明子背景可以显示背后的父背景。
8. 前者对齐容器内部行内内容；后者在适用的普通块布局及有剩余空间时分配盒子两侧外边距。
9. 字号为24px，`1em`内边距为24px；换成`1rem`后内边距为16px。
10. content-box边框外沿宽度为344px；border-box内容区宽度为256px。
11. content-box下内边距和边框会额外增加；border-box也不包括margin。
12. 题设正常流条件下可能发生正向垂直外边距折叠，结果采用较大的30px。
13. 普通非替换行内元素与替换元素的尺寸规则不同，img可采用宽高并保持固有比例。
14. 工具里的修改通常只改变当前页面临时状态，没有保存到源文件。
15. 尚未实现数据接收、处理与持久化等业务。本章的静态示例明确禁用提交，不应被描述为真实报名系统。

C.2 代码审查练习分析

`#message`要求元素具有`id="message"`，实际段落只有`class="message"`，因此没有匹配。没有其他颜色声明干预时，段落继承`#notice`设置的红色。若确实要把这类消息设为蓝色，应把选择器改成`.message`，此时段落直接采用蓝色，不再继承父颜色。

卡片为content-box，父容器内容宽度为500px，`width: 100%`得到500px的内容区，再加左右各24px内边距与1px边框，边框外沿宽度为550px。可以改成border-box，使这个声明宽度包含内边距和边框；对于本例普通正常流块盒子，也可以根据实际需要保留自动宽度。修复以后用工具核对，不只观察某一个窗口下的截图。

C.3 第5.12节的追加计算

在声明宽度240px、左右边框各2px不变的前提下，左右padding改为各30px：content-box内容区仍为240px，边框外沿为304px；border-box边框外沿仍为240px，内容区为176px。自动高度可能因为换行增加而改变。

C.4 一个建议的排错顺序

现象	首先检查	后续验证
全页接近默认外观	link路径、文件保存与加载情况	查看CSS资源能否打开
只有某个类样式失效	元素类名、点号与井号、后代关系	确认Styles中是否出现预期规则
匹配了规则却颜色不同	覆盖声明、内联样式与继承来源	临时禁用候选声明并观察
元素超过父容器	box-sizing、width、padding、border、margin	对照内容区与边框外沿尺寸
两段之间间距不符预期	默认边距、声明是否生效、折叠条件	清晰隔离示例后测量
修改刷新后失效	是否只改了开发者工具临时状态	写回源文件、保存并重新加载

参考资料

- MDN：CSS入门——规则与样式表的引入。
- MDN：选择器——常见选择条件与组合方式。
- MDN：处理样式冲突——层叠、继承与优先级，对应第4部分。
- MDN：CSS长度——px、em、rem等参照关系，对应第3.5节。
- MDN：盒模型——盒子区域与显示方式，对应第5部分。
- MDN：外边距折叠——折叠条件与边界，对应第5.11节和附录B.4。